Human-Supervised CFAR Autotuning for Maritime Sensing

Nathan Kirk*, J.C. Vaught†, Douglas Cahl‡, Yi Wang§
University of South Carolina, Columbia, SC, 29208

Maritime domain awareness, search-and-rescue, and coastal monitoring require robust radar detection under tight size, weight, power, and cost constraints. Tuning Constant False Alarm Rate (CFAR) detectors for littoral scenes is difficult due to heavy-tailed sea clutter, land/sea transitions, and shoreline multipath, yet remains essential for low-latency situational awareness. We present a human-supervised methodology and open-source browser-based tool, *CFAR Studio*, for tuning CFAR parameters of commercial-off-the-shelf X-band maritime radars. The operator annotates stationary reference objects on one or more radar frames using an interactive blob-tracing tool. The system then performs an exhaustive grid search over three CFAR variants—Cell-Averaging (CA), Greatest-Of (GO), and Smallest-Of (SO)—evaluating each parameter configuration against the annotated ground truth via an area-weighted coverage score. Multi-frame temporal stability is assessed by tracking detection consistency across sequential radar sweeps. An integral-image optimization enables the evaluation of hundreds of configurations in seconds entirely within the browser. The system is demonstrated on real X-band radar data collected with a Furuno NXT DRS4D radar and Raspberry Pi 5 at Lake Murray, South Carolina.

I. Introduction

Coastal and port approaches are operationally challenging radar environments due to strong, non-Gaussian sea clutter, land–sea transition edges, multipath from shoreline infrastructure, and mixed cooperative/non-cooperative traffic. Selecting appropriate CFAR detector parameters for these scenes is notoriously scene- and sensor-dependent: a configuration that works in open water may generate excessive false alarms near a marina, while aggressive clutter rejection near shore may mask small targets of interest. Currently, operators tune CFAR parameters by manual trial-and-error—adjusting guard cells, reference windows, and false alarm rates until the display "looks right." This process is slow, subjective, and not reproducible.

AIAA motivation. Small unmanned aircraft systems (UAS) and optionally piloted platforms are increasingly tasked with maritime domain awareness, search-and-rescue, and coastal environmental monitoring. Their radars often operate at low grazing angles in littoral airspace with tight size, weight, power, and cost (SWaP-C) constraints. A configurable CFAR stack that can be quickly tuned in-situ to a given shoreline helps enable robust on-board autonomy and low-latency situational awareness.

Contributions. This work makes the following contributions:

1) A formal scoring function for CFAR evaluation anchored to human-annotated ground truth blobs, using area-weighted coverage ratios rather than binary detection indicators.
2) An exhaustive grid search over three CFAR variants (CA, GO, SO) with integral-image loop hoisting that enables evaluation of hundreds of configurations in seconds.
3) A multi-frame temporal stability metric that quantifies detection consistency across sequential radar sweeps.
4) An open-source, fully client-side browser-based tool (*CFAR Studio*) that requires no installation, server infrastructure, or specialized hardware beyond a modern web browser.

*Undergraduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1924156.
†Graduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1537189.
‡Professor, Department of Mechanical Engineering.
§Professor, Department of Mechanical Engineering.

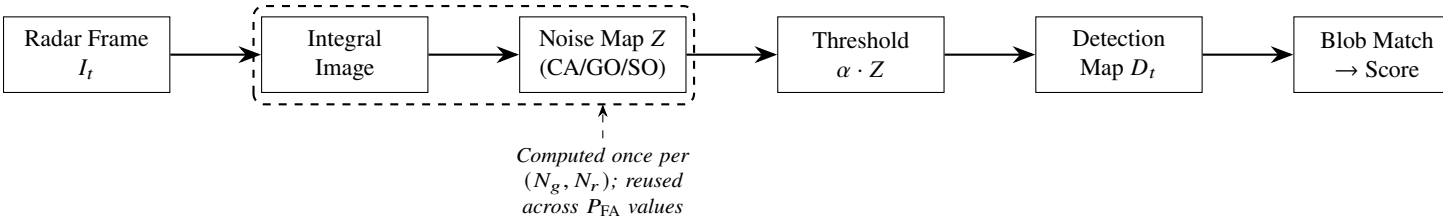

Fig. 1 CFAR detection pipeline for a single parameter configuration. The noise map depends only on guard and reference cell counts, enabling *loop hoisting*: for each (N_g, N_r) pair, the noise map is computed once and reused across all P_{FA} values, reducing redundant computation.

II. Background and Related Work

A. CFAR Detectors

The CA-CFAR family estimates background power via local training windows and compares the cell-under-test to a scaled statistic to maintain a desired probability of false alarm P_{FA} [1, 4, 10]. GO- and SO-CFAR improve robustness near clutter edges and in multi-target scenes by partitioning leading and trailing reference windows and selecting the greatest-of or smallest-of their respective noise estimates [2, 3]. Other variants—including Order-Statistic (OS-CFAR), Variability-Index, and censored/trimmed-mean CFAR—address additional heterogeneous background scenarios [5] but are beyond the scope of the current implementation.

B. Littoral Sea-Clutter Context

High-resolution maritime clutter is heavy-tailed and spatially nonhomogeneous; K-distribution and Pareto models are widely used to characterize such environments [6]. These non-Gaussian statistics motivate careful CFAR parameter selection, as training windows that span land–sea transitions or surf zones can severely bias noise estimates.

C. CA-CFAR Threshold Multiplier

For CA-CFAR with N independent, identically distributed reference cells drawn from an exponential background, the threshold multiplier α that achieves a target P_{FA} admits the closed-form expression [1, 7]:

$$\alpha = N\left(P_{\mathrm{FA}}^{-1/N} - 1\right). \tag{1}$$

This relationship is used directly in the implementation; the same α formula is applied to GO- and SO-CFAR using the effective cell count of the selected half-window.

Positioning relative to prior work. CFAR has been widely used across maritime and aerial surveillance owing to its controllable P_{FA}, local-statistics design, and suitability for embedded implementations [4–6]. Despite numerous efforts to simplify or automate CFAR tuning—ranging from vendor rule-of-thumb guides and interactive GUIs to heuristic search over thresholds for a single variant [10]—we are not aware of prior work that explicitly combines human annotation of stationary reference objects with exhaustive multi-variant parameter search and area-based scoring in a browser-deployable tool. Our approach uses human-provided ground truth to define an objective function while leveraging computational optimization to efficiently explore the parameter space.

III. Problem Formulation

Let $I_t \in \mathbb{R}^{M\times N}$ denote radar intensity frames in range–azimuth coordinates, $t = 1, \ldots, T$. A CFAR variant $v \in \{\mathrm{CA}, \mathrm{GO}, \mathrm{SO}\}$ with parameter vector $\boldsymbol{\theta} = (N_g, N_r, P_{\mathrm{FA}})$—guard cells, reference cells, and probability of false alarm—is applied to each frame to produce a binary detection map $D_t(v, \boldsymbol{\theta}) \in \{0, 1\}^{M\times N}$.

A. CFAR Detection

All three variants share the same two-dimensional sliding-window structure. For each pixel (i, j) in frame I_t, a guard region of radius N_g excludes the cell-under-test and its immediate neighbors; a reference annulus of width N_r

surrounds the guard region. An integral image (summed area table) [9] enables $O(1)$ computation of the sum over any rectangular region. The noise estimate $Z_{i,j}$ is computed as follows:

CA-CFAR. The mean of all $N = (2R_o + 1)^2 - (2R_i + 1)^2$ reference cells, where $R_o = N_g + N_r$ and $R_i = N_g$:

$$Z_{i,j}^{\mathrm{CA}} = \frac{1}{N} \left(S_{\mathrm{outer}} - S_{\mathrm{inner}} \right), \tag{2}$$

where S_{outer} and S_{inner} are the summed-area-table queries over the outer and inner rectangles, respectively.

GO-CFAR. The reference annulus is split into leading and trailing half-windows (above and below the cell-under-test). Let $\bar{Z}_1$ and $\bar{Z}_2$ denote the mean power of each half-window, each containing N_{half} cells:

$$Z_{i,j}^{\mathrm{GO}} = \max(\bar{Z}_1, \bar{Z}_2). \tag{3}$$

SO-CFAR. The smallest-of selection:

$$Z_{i,j}^{\mathrm{SO}} = \min(\bar{Z}_1, \bar{Z}_2). \tag{4}$$

For GO- and SO-CFAR, the effective reference cell count used for α computation is N_{half} rather than the full annulus count.

A detection is declared when the cell-under-test exceeds the scaled noise estimate:

$$D_t(i,j) = \mathbb{1}\left[I_t(i,j) > \alpha \cdot Z_{i,j} \right], \tag{5}$$

where α is given by Eq. (1) using the appropriate cell count.

B. Ground Truth Annotation

The operator annotates stationary reference objects (e.g., navigation buoys, towers, dock structures) on the radar PPI display. When the operator clicks on a detected pixel, an 8-connected flood-fill algorithm traces the contiguous detection blob $\mathcal{B}_k$, collecting all connected pixels into a set of indices. This "magic wand" approach captures the exact shape and area of each reference object as it appears in the current CFAR detection, rather than requiring the operator to draw bounding boxes or estimate target extent. The resulting blob pixel set $\mathcal{B}_k = \{p_1, p_2, \ldots, p_{|\mathcal{B}_k|}\}$ serves as the ground truth reference for scoring.

C. Area-Weighted Scoring Function

For each ground truth blob $\mathcal{B}_k$ and a candidate detection map D_t, we compute the *area coverage ratio*:

$$\rho_k = \frac{\sum_{p \in \mathcal{B}_k} D_t(p)}{|\mathcal{B}_k|}. \tag{6}$$

This ratio measures what fraction of the ground truth blob's pixels are detected by the candidate configuration. The coverage ratio is mapped to a discrete score via a piecewise function that rewards configurations capturing the full target shape:

$$s_k = \begin{cases} 1.00 & \text{if } 0.8 \le \rho_k \le 1.1, \\ 0.75 & \text{if } 0.6 \le \rho_k < 0.8, \\ 0.50 & \text{if } 0.4 \le \rho_k < 0.6, \\ 0.25 & \text{if } 0.2 \le \rho_k < 0.4, \\ 0.00 & \text{if } \rho_k < 0.2. \end{cases} \tag{7}$$

The upper bound of 1.1 accommodates slight detection expansion beyond the annotated blob boundary, which is common when a slightly more sensitive configuration captures peripheral pixels. This area-based approach provides substantially more nuance than a binary "detected/not-detected" metric: a configuration that detects only a small fragment of a large target is appropriately penalized, while one that captures the full target shape is rewarded.

The per-frame score across all K ground truth targets combines the worst-case and average target scores:

$$S_{\mathrm{frame}} = 0.7 \cdot \min_{k \in [K]} s_k + 0.3 \cdot \frac{1}{K} \sum_{k=1}^{K} s_k. \tag{8}$$

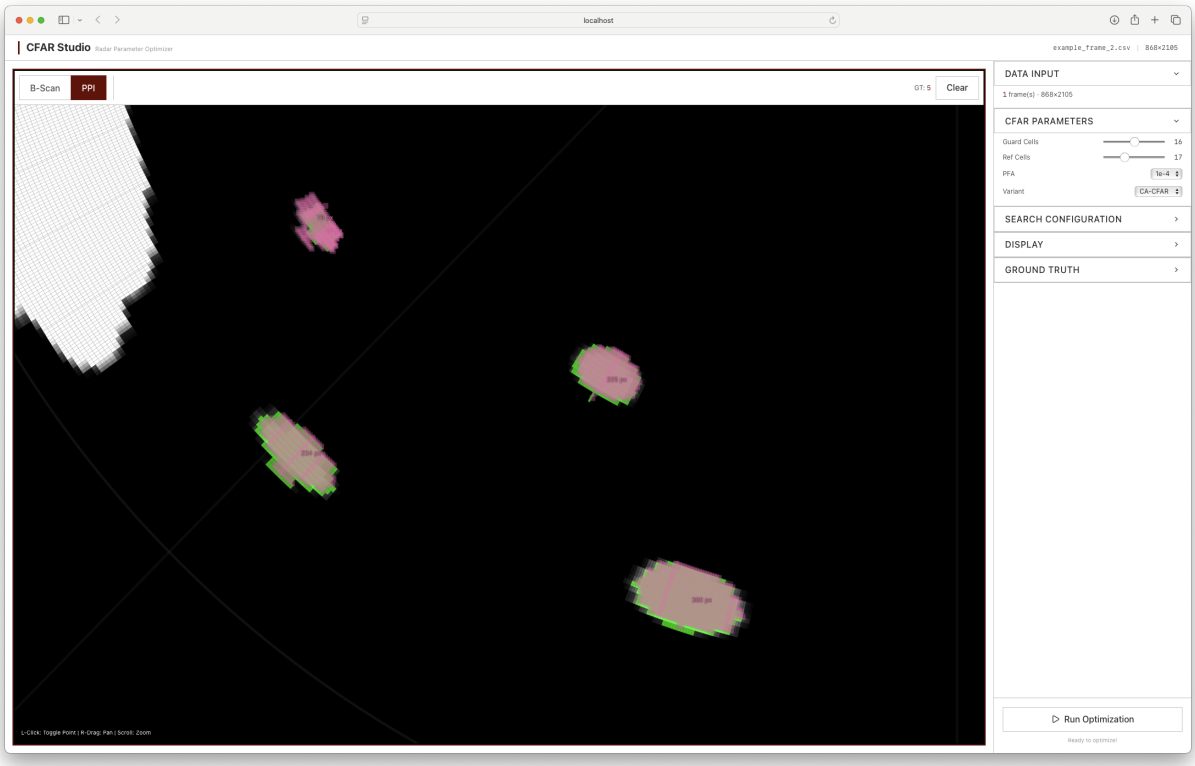

Fig. 2 Close-up of ground truth annotations on the PPI display. The operator clicks on a detected target; an 8-connected flood fill traces the contiguous detection blob, capturing its exact pixel footprint as the reference shape for scoring.

The heavy weighting on the minimum score (0.7) enforces a critical constraint: a configuration that completely misses even one annotated target is severely penalized regardless of how well it detects the others. This prevents the optimizer from recommending configurations that sacrifice coverage of difficult targets in favor of easy ones.

False alarm count is computed as the number of detected pixels outside all ground truth blobs:

$$\mathrm{FP} = \sum_{t=1}^{T} \left(\sum_{(i,j)} D_t(i,j) - \sum_{k=1}^{K} \sum_{p \in \mathcal{B}_k} D_t(p) \right). \tag{9}$$

D. Multi-Frame Temporal Stability

When multiple sequential frames are available, temporal stability quantifies how consistently each target is detected across the frame sequence. For each ground truth target k, let c_k be the number of frames in which it is detected (score > 0). The stability metric is

$$\text{Stability} = \frac{1}{K} \sum_{k=1}^{K} \frac{c_k}{T}. \tag{10}$$

A stability of 1.0 indicates all targets are detected in every frame; low stability reveals configurations that are marginally sensitive and produce intermittent detections.

E. Grid Search with Integral-Image Loop Hoisting

The parameter space is defined by discrete ranges for guard cells N_g, reference cells N_r, and a set of P_{FA} values. All combinations are evaluated exhaustively.

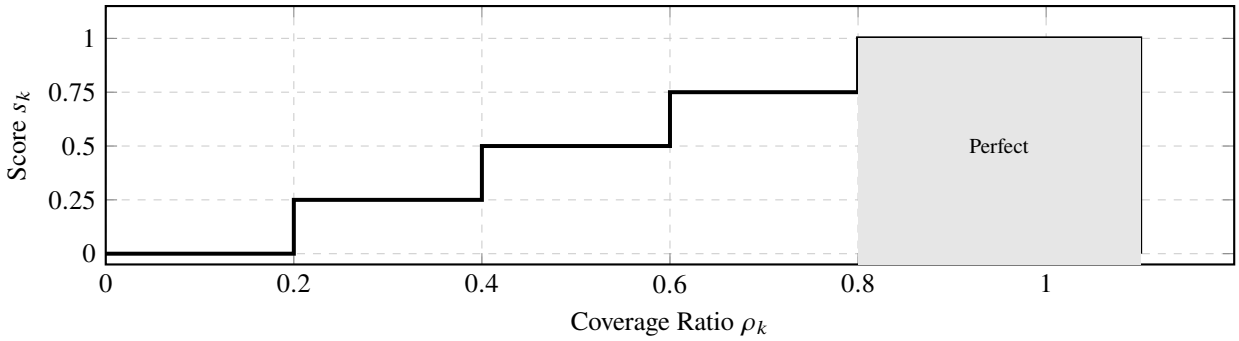

Fig. 3 Scoring function mapping area coverage ratio ρ_k to discrete score s_k. Configurations that capture 80–110% of the ground truth blob area receive a perfect score; partial detections are penalized proportionally.

Algorithm 1 Grid Search with Loop Hoisting

Input: frames $\{I_t\}_{t=1}^{T}$, ground truth blobs $\{\mathcal{B}_k\}$, variant v, parameter ranges
Pre-compute integral images for all frames
for each (N_g, N_r) in parameter grid **do**
  Compute noise map Z for all frames ▷ *Expensive step*
  for each P_{FA} value **do**
    $\alpha \leftarrow N_{\text{eff}}(P_{\text{FA}}^{-1/N_{\text{eff}}} - 1)$
    Apply threshold: $D_t(i,j) \leftarrow \mathbb{1}[I_t(i,j) > \alpha \cdot Z_{i,j}]$
    Compute coverage scores $\{s_k\}$, frame score, false alarms, stability
  end for
end for
Return all results sorted by score

The key computational optimization exploits the fact that the noise map Z depends only on (N_g, N_r, v) and not on P_{FA}. By restructuring the search loop to iterate over (N_g, N_r) in the outer loop and P_{FA} in the inner loop, the expensive noise map computation is performed once per (N_g, N_r) pair and reused across all P_{FA} values (Fig. 1). Within the inner loop, only the scalar multiplication $\alpha \cdot Z_{i,j}$ and comparison in Eq. (5) are needed. This *loop hoisting* reduces the total number of integral-image queries by a factor equal to the number of P_{FA} values tested.

IV. System Architecture

CFAR Studio is implemented as a fully client-side web application. All computation occurs in the user's browser with no server-side processing, no data upload, and no installation required. The software stack consists of:

- **Application framework:** React 19 with TypeScript, built with Vite.
- **Visualization:** Plotly.js for interactive PPI (Plan Position Indicator) and B-scan displays with multiple colormaps and zoom/pan controls.
- **Data ingestion:** PapaParse for parsing CSV files exported from the radar digitizer.
- **Optimization engine:** A dedicated Web Worker thread that performs the grid search without blocking the user interface.

A. Data Ingestion

Radar data is ingested from CSV files containing range–azimuth intensity matrices. Each CSV row corresponds to one azimuth spoke, with columns representing range bins. A companion azimuth array specifies the angular position of each spoke. The system supports multi-frame loading, enabling temporal analysis across sequential radar sweeps.

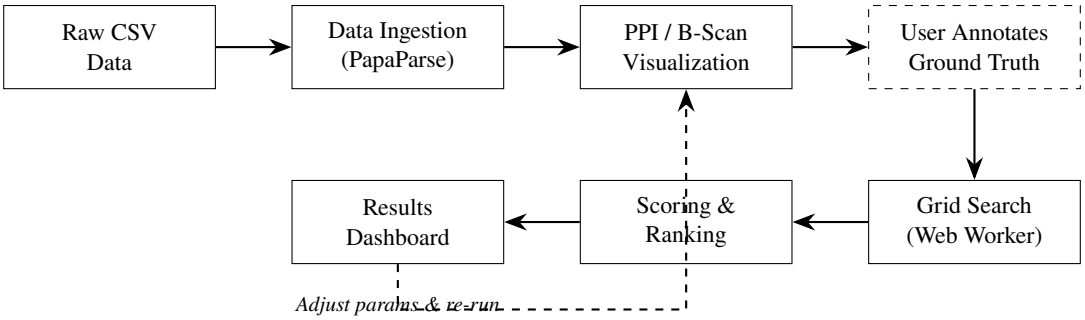

Fig. 4 CFAR Studio system pipeline. Dashed box indicates user interaction; dashed arrow shows the iterative feedback loop where the operator can adjust parameters and re-run optimization after reviewing results.

B. Visualization

The primary display is a PPI view rendered in Cartesian coordinates, with options for multiple colormaps (grayscale, amber, green, inferno, ice). CFAR detections are overlaid as colored highlights, and ground truth annotations are displayed as distinct markers. A B-scan view (Fig. 6) provides an alternative range–azimuth representation useful for examining clutter structure and detection behavior along individual spokes.

C. Ground Truth Workflow

The annotation workflow proceeds as follows: (1) the operator loads radar CSV data and selects CFAR parameters for an initial preview detection; (2) on the PPI display, the operator clicks on detected targets representing known stationary objects; (3) the system performs an 8-connected flood fill from the click point, tracing the contiguous detection blob and recording all constituent pixel indices as the ground truth reference; (4) the blob pixel count is displayed as a size indicator. Ground truth annotations persist across frames, with the assumption that the radar is stationary and reference objects do not move.

D. Optimization Engine

When the operator initiates optimization, the parameter ranges (guard cells, reference cells, P_{FA} values) and CFAR variant are dispatched to a Web Worker. The worker executes Algorithm 1, posting progress updates at regular intervals. Upon completion, results are returned to the main thread for visualization in a multi-dimensional results dashboard (Fig. 7), including sortable tables, parameter trend plots, and interactive filtering by score, false alarm count, and stability.

V. Experimental Setup and Results

A. Hardware and Data Collection

Radar data was collected using a Furuno NXT DRS4D X-band (9.41 GHz) solid-state maritime radar connected to a Raspberry Pi 5 single-board computer for raw data digitization and recording. The radar was operated in a shore-based configuration at Lake Murray, Columbia, South Carolina. Lake Murray provides a mixed littoral environment with open water, shoreline structures, navigation buoys, and dock infrastructure—representative targets for CFAR tuning evaluation.

Raw radar returns were captured as CSV files, with each file representing one complete antenna rotation (360° sweep). Multiple sequential sweeps were recorded to enable multi-frame temporal analysis.

B. Ground Truth Selection

Stationary reference objects were annotated using the blob-tracing tool described in Section III.B. Targets included navigation buoys, fixed dock structures, and shoreline features that produce consistent radar returns across multiple sweeps. For each target, the operator clicked on the detection under an initial CFAR configuration, and the flood-fill algorithm captured the blob footprint as the ground truth reference.

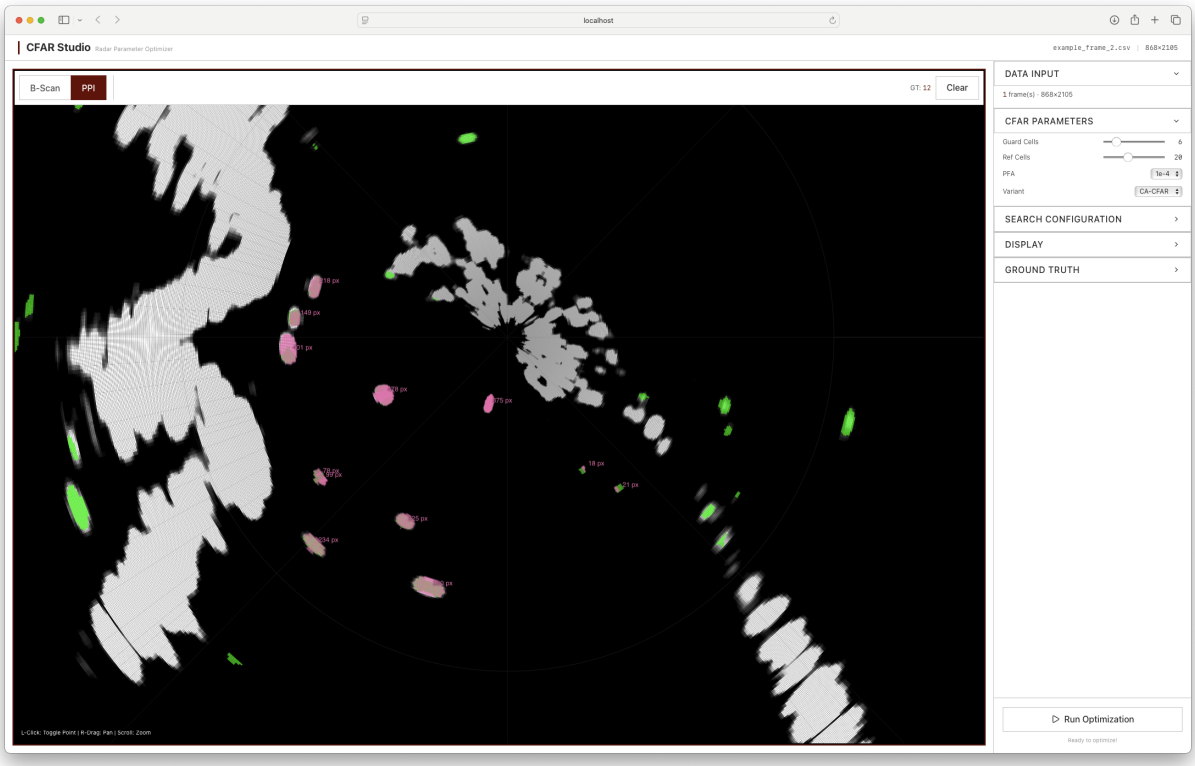

Fig. 5 PPI display showing raw radar intensity with CFAR detections (highlighted) and annotated ground truth blobs. The overlay enables visual comparison of detection coverage against known targets.

C. Optimization Configuration

The grid search was configured with the following parameter ranges:

- Guard cells: $N_g \in [1, 10]$, step 1
- Reference cells: $N_r \in [5, 30]$, step 5
- P_{FA}: logarithmically spaced values in $[10^{-6}, 10^{-2}]$
- Variants: CA, GO, SO (each optimized independently)

A typical grid of approximately 500 parameter configurations completes in under 10 seconds on a modern laptop browser, owing to the integral-image loop hoisting described in Section III.E.

D. Results

The optimization results reveal several general trends across the tested CFAR variants:

Guard cells. Larger guard cell counts ($N_g \geq 3$) consistently improve scores for extended targets (e.g., dock structures), as they prevent the target's own return from biasing the noise estimate. For point-like targets (buoys), guard cell sensitivity is lower.

Reference cells. Moderate reference window sizes ($N_r \in [10, 20]$) provide the best trade-off between noise estimation accuracy and spatial homogeneity. Very large reference windows ($N_r > 25$) degrade performance near land–sea transitions where the training window spans heterogeneous clutter regions.

P_{FA} sensitivity. Lower P_{FA} values (more conservative) reduce false alarms but risk missing weaker targets. The optimal P_{FA} depends strongly on the scene: open-water targets tolerate lower P_{FA}, while targets near clutter edges benefit from moderate P_{FA} to maintain detection.

Variant comparison. GO-CFAR tends to produce fewer false alarms near clutter edges at the cost of slightly reduced detection sensitivity, consistent with its conservative (maximum) noise selection. SO-CFAR provides higher detection rates but generates more false alarms in heterogeneous regions. CA-CFAR offers a middle ground and often produces the highest overall scores in relatively homogeneous scenes.

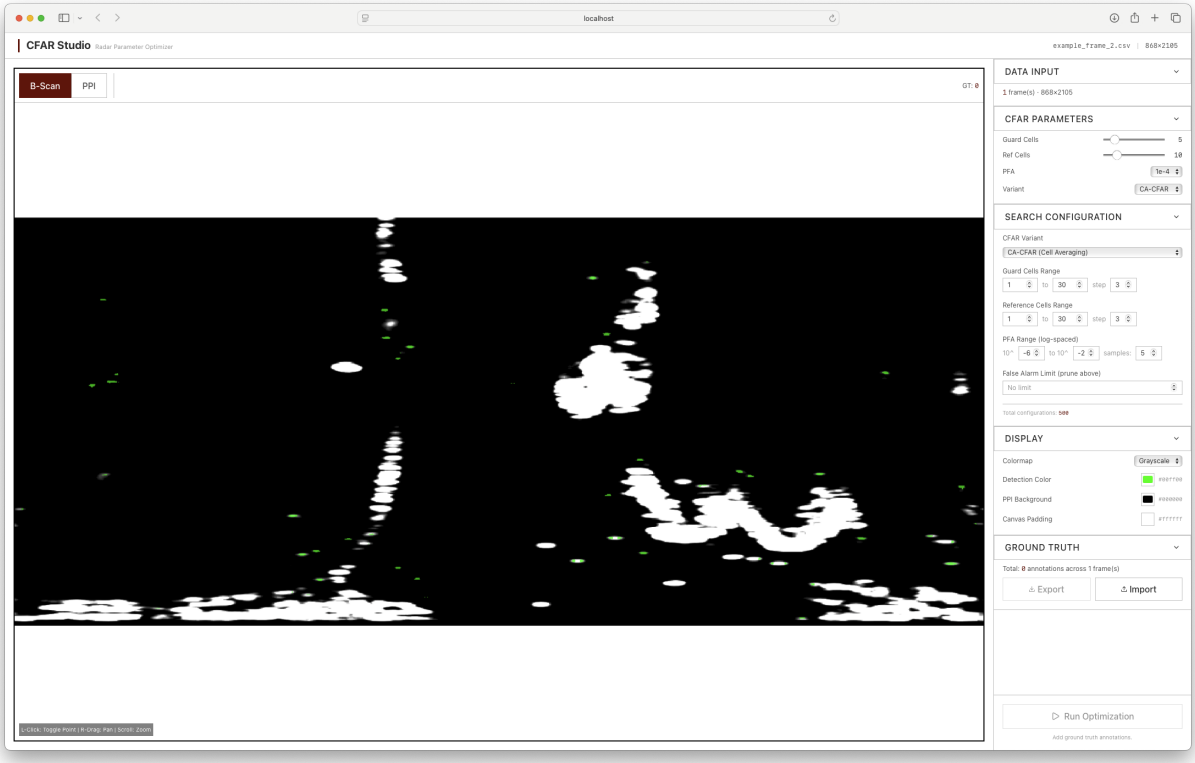

Fig. 6 B-scan (range vs. azimuth) view with CFAR detections overlaid. This representation reveals clutter structure along individual radar spokes and aids in understanding detection behavior near land–sea transitions.

Temporal stability. Across multi-frame sequences, the top-scoring configurations for each variant typically achieve stability values above 0.9, indicating consistent detection of annotated targets across sequential sweeps. Configurations with marginal scores (near the detection threshold) show substantially lower stability, confirming that the multi-frame metric effectively identifies robust parameter settings.

VI. Discussion and Limitations

Stationary reference assumption. The method assumes quasi-stationary reference objects and a fixed radar platform, so that ground truth annotations remain valid across frames. Extension to moving targets or mobile radar platforms would require frame-to-frame registration or track-before-detect association.

Browser constraints. Running entirely in the browser imposes limitations: single-threaded JavaScript execution (mitigated by Web Worker offloading), no GPU acceleration, and a practical file size limit of approximately 50 MB per CSV. For the dataset sizes typical of shore-based X-band radar sweeps, these constraints are not limiting.

Grid search vs. sophisticated optimization. An exhaustive grid search is computationally feasible for the three-variant, three-parameter space addressed here (guard, reference, P_{FA}), with typical grids containing a few hundred configurations evaluable in seconds. More sophisticated approaches (Bayesian optimization, evolutionary algorithms) would become necessary if the parameter space were expanded to include additional CFAR variants with extra parameters (e.g., OS-CFAR order statistic k, censoring fractions).

Area-based vs. binary scoring. The area-weighted coverage score (Eq. 7) provides substantially more information than a binary detected/not-detected metric. A configuration that detects only a small fragment of a large target receives a low score, whereas binary scoring would assign full credit. This distinction is important for targets where shape fidelity affects downstream processing (e.g., target classification or extent estimation).

Current limitations. The tool supports three CFAR variants (CA, GO, SO); incorporating OS-CFAR or censored variants would require sorting operations that increase per-pixel cost. Data ingestion is limited to the CSV format

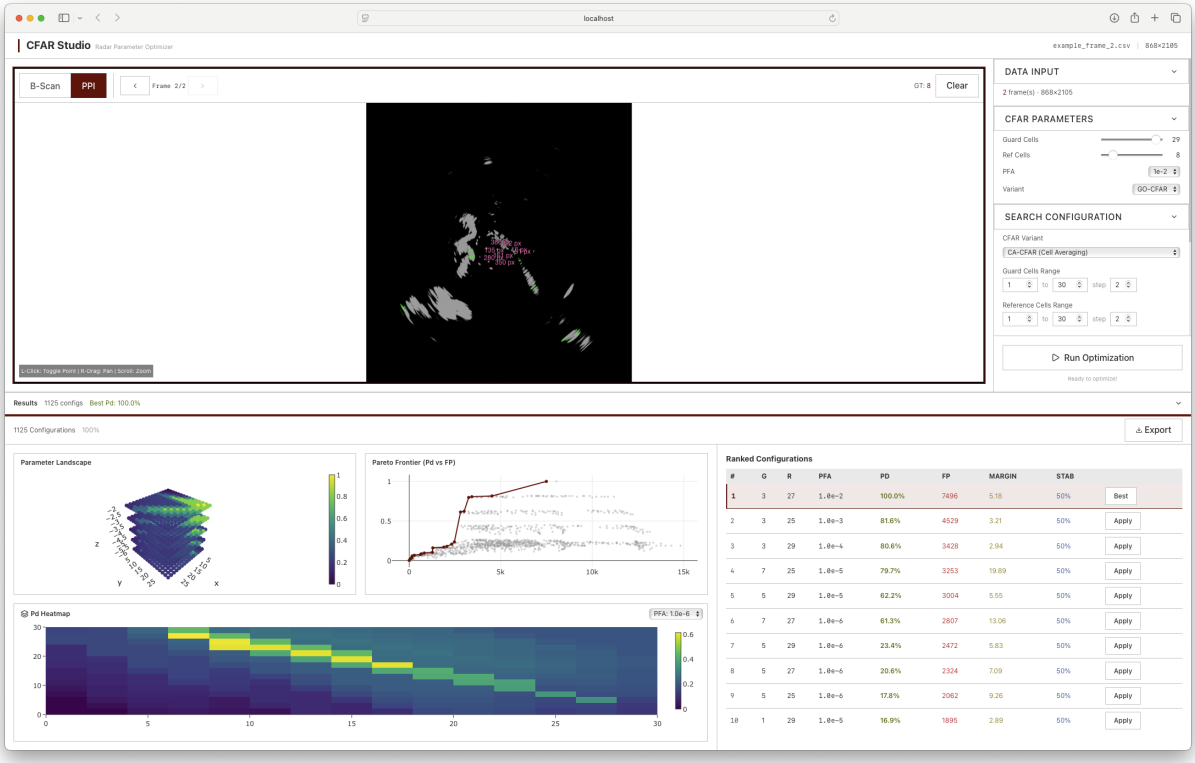

Fig. 7 Full CFAR Studio interface showing the PPI radar display (left), parameter controls (center), and optimization results dashboard (right) with score rankings and parameter trend visualizations.

produced by the Furuno/RPi5 digitization pipeline; support for additional radar formats would require parser extensions.

VII. AIAA Relevance and Use Cases

For maritime UAS, rapidly configurable CFAR improves: (1) on-board detection of fixed navigation aids during autonomous coastal mapping; (2) port approach monitoring and deconfliction; (3) search-and-rescue in surf zones where sea spikes create false alarms. The browser-based architecture requires no software installation, no specialized computing hardware, and no network connectivity for operation—the operator simply opens CFAR Studio in a laptop browser alongside the radar feed. This makes the tool immediately deployable in field settings where installing specialized signal processing software is impractical.

VIII. Conclusion

We presented CFAR Studio, an open-source, browser-based tool for human-supervised CFAR parameter tuning in littoral radar environments. The system enables an operator to annotate stationary reference objects via blob tracing, then exhaustively searches over three CFAR variants (CA, GO, SO) and their parameter spaces using an integral-image-accelerated grid search. An area-weighted scoring function evaluates each configuration against the annotated ground truth, penalizing partial detections and rewarding full-shape coverage. Multi-frame temporal stability identifies parameter settings that produce consistent detections across sequential radar sweeps.

Demonstrated on real X-band radar data from a Furuno NXT DRS4D at Lake Murray, South Carolina, the tool evaluates hundreds of configurations in seconds within a standard web browser. The results identify scene-specific parameter trends and variant trade-offs that would be difficult to discover through manual trial-and-error tuning. The fully client-side architecture, requiring no installation or server infrastructure, makes the tool suitable for rapid field deployment in maritime UAS and coastal monitoring applications.

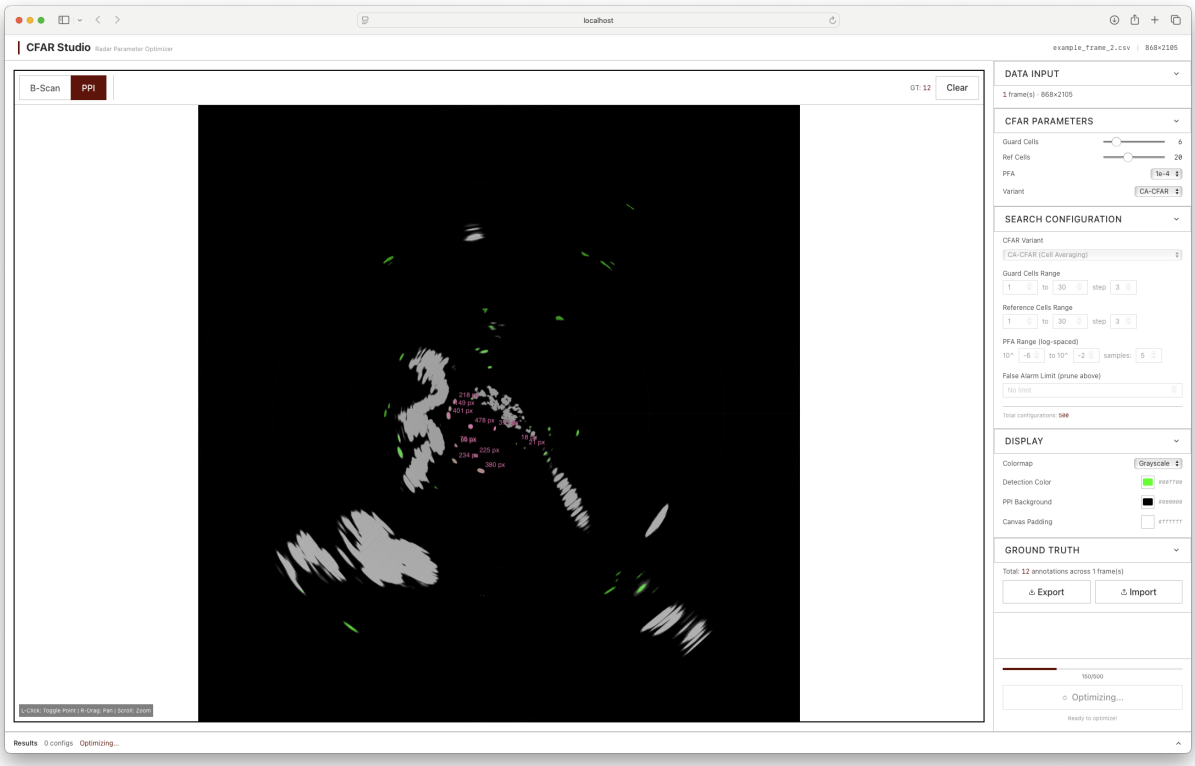

Fig. 8 CFAR Studio during an active optimization run. The progress indicator shows the grid search advancing through parameter configurations while the PPI display remains interactive.

The source code is available at https://github.com/IMSEL-Lab/cfar-studio.

A. Software Stack

Table 1 CFAR Studio technology stack.

Component	Technology
Application Framework	React 19 + TypeScript 5.9
Build Tool	Vite 7.2
Visualization	Plotly.js 3.3
CSV Parsing	PapaParse 5.5
Styling	TailwindCSS
Icons	Lucide React
Optimization	Web Worker (dedicated thread)

References

[1] H. M. Finn and R. S. Johnson, "Adaptive detection mode with threshold control as a function of spatially sampled clutter level estimates," *RCA Review*, vol. 29, pp. 414–464, Sept. 1968.

[2] V. G. Hansen and J. H. Ward, “Detection performance of the cell averaging log/CFAR receiver,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-8, no. 5, pp. 648–654, Sept. 1972.

[3] G. V. Trunk, “Range resolution of targets using automatic detectors,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-14, no. 5, pp. 750–755, 1978.

[4] P. P. Gandhi and S. A. Kassam, “Analysis of CFAR processors in nonhomogeneous background,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 24, no. 4, pp. 427–445, Jul. 1988.

[5] A. Farina, “A review of CFAR detection techniques in radar systems,” in *DSP for Radar and Communications*, 2005, pp. 1–42.

[6] K. D. Ward, R. J. A. Tough, and S. Watts, *Sea Clutter: Scattering, the K Distribution and Radar Performance*. IET, 2006.

[7] M. A. Richards, *Fundamentals of Radar Signal Processing*. McGraw-Hill, 2005.

[8] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[9] F. C. Crow, “Summed-area tables for texture mapping,” in *Proc. ACM SIGGRAPH*, vol. 18, no. 3, pp. 207–212, 1984.

[10] MathWorks, “Constant False Alarm Rate (CFAR) Detection,” online documentation, accessed Nov. 2025.